



SIMATS Online Programming Lab

JAVA PROGRAMMING



KADIRI ASMITHA
192425038RD

Multithreading Practice - CO3

[← Back](#)

QUESTIONS

Q1

Multiple Threads Using a Single Class

10 marks

**Q2**

Multiple thread classes

20 marks

**Q3**

Thread Synchronization

10 marks



3/3 answered

Q1 Multiple Threads Using a Single Class 10 marks

Topic: Thread class, multiple threads, if-else

Question:

Write a Java program using a single thread class to create three threads named Prime, Armstrong, and Reverse.

The program should contain a common run() method. Based on the thread name, use if-else statements to perform the following operations on a given number:

Prime → Check whether the number is prime.

Armstrong → Check whether the number is an Armstrong number.

Reverse → Find the reverse of the number.

Create and start all three threads from the main() method.

Input: 153

Expected Output:

Prime Thread: 153 is not a prime number

Armstrong Thread: 153 is an Armstrong number

Reverse Thread: Reverse of 153 = 351

Your Code

```
1 import java.util.Scanner;
2
3 public class ThreadA extends Thread {
4
5     int n;
6     String name;
7
8     ThreadA(String name, int n) {
9         this.name = name;
10        this.n = n;
11    }
12
13    public void Prime(int n) {
14        int count = 0;
15
16        for (int i = 1; i <= n; i++) {
17            if (n % i == 0) {
18                count++;
19            }
20        }
21
22        if (count == 2)
23            System.out.println("Prime Thread");
24        else
25            System.out.println("Not Prime Thread");
26    }
27
28    public void Armstrong(int n) {
29        int temp = n;
30        int sum = 0;
31
32        for (; n > 0; n = n / 10) {
33            int digit = n % 10;
34            sum = sum + digit * digit * digit;
35        }
36
37        if (sum == temp)
38            System.out.println("Armstrong Number");
39        else
40            System.out.println("Not Armstrong Number");
41    }
42
43    public void Reverse(int n) {
44        int temp = n;
45        int rev = 0;
46
47        for (; n > 0; n = n / 10) {
48            int digit = n % 10;
```

```
49         rev = rev * 10 + digit;
50     }
51
52     System.out.println("Reverse Thread");
53 }
54
55 public void run() {
56
57     if (name.equals("Prime"))
58         Prime(n);
59
60     else if (name.equals("Armstrong"))
61         Armstrong(n);
62
63     else if (name.equals("Reverse"))
64         Reverse(n);
65 }
66
67 public static void main(String[] args)
68
69     Scanner sc = new Scanner(System.in);
70
71     if (sc.hasNextInt()) {
72         int n = sc.nextInt();
73
74         ThreadA t1 = new ThreadA("Prime");
75         ThreadA t2 = new ThreadA("Armstrong");
76         ThreadA t3 = new ThreadA("Reverse");
77
78         t1.start();
79         t1.join();
80
81         t2.start();
82         t2.join();
83
84         t3.start();
85         t3.join();
86     }
87     sc.close();
88 }
89 }
```



Input

153



Output

```
Prime Thread: 153 is not a prime number  
Armstrong Thread: 153 is an Armstrong number  
Reverse Thread: Reverse of 153 = 351
```

Visible Test Cases

Test Case 1



Test Case 2



Test Case 3



10.00 / 10 marks

Submitted